# Polymorphism

The word itself comes from Greek, meaning **"many forms"** (poly = many, morph = form). It is the ability of the same function or object to behave differently based on the input or the class that calls it.

**1. Compile-Time Polymorphism (Static Binding):** The compiler determines which function to call during compilation.

- Function Overloading
- Operator Overloading

**2. Run-Time Polymorphism (Dynamic Binding):** The decision of which function to execute is made while the program is running.

- Function Overriding
- Virtual Functions

## Function Overloading

Allows multiple **functions** within the **same scope** to share the **same name** but have **different** parameters. It allows a single function name to perform different tasks based on the type or number of arguments it receives.

**How it works:** The compiler distinguishes between functions by examining their **signatures** (the list of parameter types).

### The Golden Rule of Overloading

To overload a function, the **function signature** must change. The signature includes a **number** of parameters. The **data type** of the parameters. The **order** of the parameters. (Note: Changing only the return type is not enough.)

```
class Car {
      // Version 1: No parameters
      void accelerate() { speed += 10; }

      // Version 2: One integer parameter
      void accelerate(int boost) { speed += boost; }

      // Version 3: One double parameter
      void accelerate(double factor) { speed *= factor; }
  };
```

**Why use Function Overloading?**

- **Cleaner APIs:** Developers don't need to memorize a dozen differently named methods for the same fundamental action.
- **Flexibility:** It allows a single operation (like calculating an area, formatting a date) to accept various forms of input gracefully.

## Operator Overloading

Allows you to redefine how standard operators **(like +, -, ==, or <<)** work with user-defined objects. It gives custom meaning to operators so you can perform operations on objects just like you do with built-in types (int, float).

### The Concept

Out of the box, the compiler knows exactly what the **+** operator should do when it sees two integers **(5 + 5 = 10).** It also often knows what to do with strings **("Hello" + " World" = "Hello World").**

But what if you create a custom **Vector** class, or a **BankAccount** class? The compiler has no idea what it means to "add" two bank accounts together. Operator overloading allows you to explicitly teach the compiler how to apply these standard symbols to your objects.

### Why use it?

The primary reason to use operator overloading is to make your code much more readable and intuitive. Imagine you are building a game and need to add the **X** and **Y** coordinates of two **2D** vectors.

- **Without operator overloading:** You have to create an add() method. Your code looks like this:
  **Vector3 = Vector1.add(Vector2)**
- **With operator overloading:** Your code looks like standard math: **Vector3 = Vector1 + Vector2**

### Key Rules for Operator Overloading

1. You cannot create **new** operators (like **); you can only redefine existing ones.
2. You cannot change the **precedence** or **associativity** of operators.
3. At least one operand must be a user-defined type (class or struct).

```
class Vector {

  int x, y;
  Vector(int x, int y) : x(x), y(y) {}

  // We are redefining what '+' means for the Vector class.
  Vector operator+(const Vector &other) {
    // Add the 'x' of the first vector to the 'x' of the second, same for 'y'
    return Vector(this->x + other.x, this->y + other.y);
  }
};

  // Usage:
  Vector v1(2, 3);
  Vector v2(4, 5);
  // Now we can use the '+' sign cleanly!
  Vector v3 = v1 + v2;
```

## Function Overriding

**Function Overriding** occurs when a **child class** provides a specific implementation of a method already defined in its **parent class**. It allows a child class to change the behavior of an inherited method to better suit its needs.

### The Golden Rules of Overriding

1. It must have the exact same name.
2. It must have the exact same parameters (signature).
3. It must have the same return type.
4. There must be an "Is-A" (inheritance) relationship between the two classes.

**Runtime Decision:** In C++, you use the **virtual** keyword in the parent class to ensure the correct version is called at runtime when using pointers.

```
class Car {
public:
  // Virtual keyword allows this to be overridden
  virtual void fuelType() { cout << "Uses generic fuel" << endl; }
};

class Tesla : public Car {
public:
  // Overriding the parent's behavior
  void fuelType() override { cout << "Uses Electricity" << endl; }
};
```

### The Real Magic: Dynamic Binding

The code above is simple, but it doesn't show the true power of **overriding**. The real magic happens when you treat a **child object** as if it were a **parent object**.

Imagine you create a pointer to a **Car**, but you actually hand it a **Tesla** object:

```
Car *myCar = new Tesla();
myCar->fuelType();
```

Because of **overriding** (and the **virtual** keyword in C++), the program doesn't just look at the **Car** label and make it use generic fuel. It waits until the exact moment the code runs, looks at the actual object sitting in memory (the **Tesla**), and uses electricity.